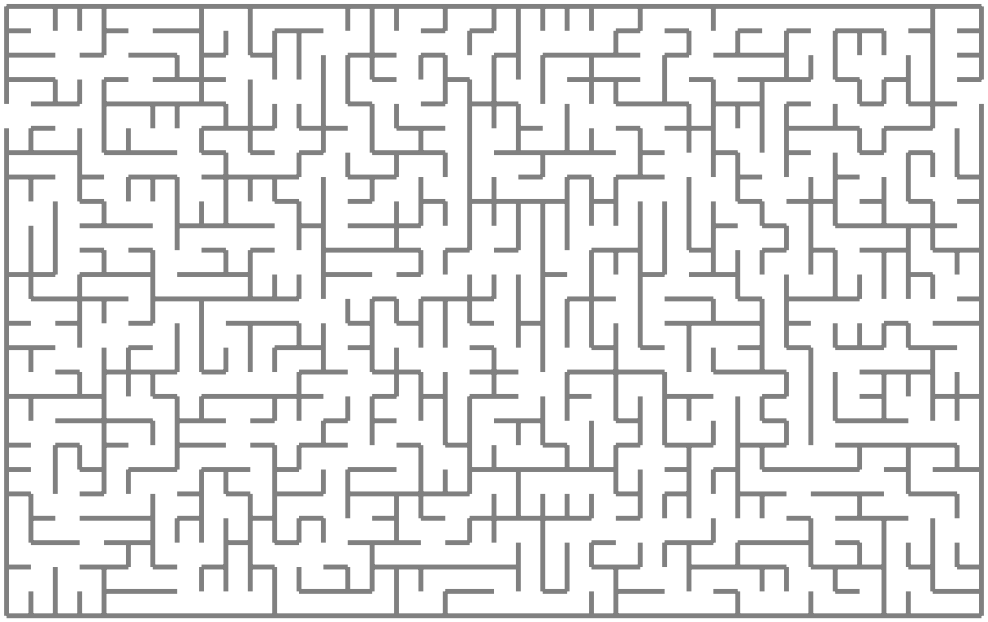
Higher order pattern unification using scope graphs

Version of April 15, 2026



Tudor Andrei

Higher order pattern unification using scope graphs

THESIS

submitted in partial fulfillment of the
requirements for the degree of

MASTER OF SCIENCE

in

COMPUTER SCIENCE

by

Tudor Andrei
born in Bucharest, Romania

Programming Languages Group
Department of Software Technology
Faculty EEMCS, Delft University of Technology
Delft, the Netherlands
www.ewi.tudelft.nl

Cover picture: Random maze.

Higher order pattern unification using scope graphs

Author: Tudor Andrei
Student id: 5495822
Email: T.Andrei@student.tudelft.nl

Abstract

This work researches the possibility of using higher order unification in a typechecker that uses scope graphs. Scope graphs are a novel data structure for name resolution created by Statix. The Statix language is part of the language workbench Spoofax, created at TuDelft, and enables users to write a typechecker for their language. Out of the box, Statix can do first order unification between terms from the user's language which is enough to typecheck the majority of type system. However, dependently typed languages present a bigger challenge, which arises from type checking definitions with implicit arguments or pattern matching. We will propose a new unification algorithm for Statix, that is able to unify higher order terms (lambda functions) while using scope graphs for name resolution.

Thesis Committee:

Chair:	Prof. dr. C. Hair, Faculty EEMCS, TU Delft
Committee Member:	Dr. A. Bee, Faculty EEMCS, TU Delft
Committee Member:	Dr. C. Dee, Faculty EEMCS, TU Delft
University Supervisor:	Ir. E. Ef, Faculty EEMCS, TU Delft

Preface

Preface here.

Tudor Andrei
Delft, the Netherlands
April 15, 2026

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Having a strong type system has become a defining characteristic of many modern programming languages. Beyond basic error detection, expressive type systems allow developers to encode invariants directly into the structure of their programs, shifting entire classes of bugs from runtime to compile time. Types serve as a precise form of documentation, making assumptions and guarantees explicit and enforceable.

Dependent type systems represent a significant step in this progression. Positioned at the top of the lambda cube, they extend conventional type systems by allowing types to depend on values. This added expressiveness enables the encoding of precise specifications. For instance, types that capture the exact length of a list or enforce domain-specific constraints. Besides the regular function type $A \to B$, this calculus introduces Π-types of the form: $(x : A) \to B$, where x can appear in the definition of B. x is only known when we apply the function on an argument. The actual value of the argument should be substituted for x in B. Additionally, through the Curry-Howard correspondence (Reis n.d.), such type systems blur the lines between programs and proofs: writing a well-typed program becomes equivalent to constructing a proof of a logical proposition. As a result, languages with dependent types can function as proof assistants for propositional logic with quantifiers. Practical realizations of these ideas can be seen in systems such as Lean, Agda, and Idris 2, where programming and formal verification are tightly integrated. For a dependently typed language to be considered practical, it needs to perform as much type inference as possible. Consider the function to add an element to the head of a list in a dependent type setting where all types are explicit:

$$cons :: (A : Set) \to (n : Int) \to A \to Vec\ A\ n \to Vec\ A\ (n + 1)$$

$cons$ is polymorphic with respect to the type of the list and the length of the list. Providing an explicit argument for the type and lenght of the list everytime the function is called is very cumbersome. To this end, production implementations manage to infer some of the types and values, making them implicit (they appear in the signature but can be omitted on application). Inferring implicit arguments is a big challenge for dependent type checkers and requires higher order (pattern) unification (Johansson and Lloyd n.d.) between terms that can be either values or types.

Language workbenches are environments designed to support the systematic development of programming languages (Erdweg et al. 2015) by providing reusable infrastructure for common compiler and tooling concerns. They exist primarily to reduce the engineering overhead associated with implementing languages from scratch, particularly for very common features such as parsing, name binding, type checking, and editor support. By elevating these concerns to declarative specifications, language workbenches allow language designers to focus on semantics rather than implementation details. Language workbenches enable rapid prototyping, facilitate experimentation with advanced type systems, and promote correctness through well-defined meta-languages and tooling. As an example, in the

case of dependent type languages, building all the necessary tooling from scratch is a difficult task. Type checking in such systems often involves combining mechanisms like higher-order unification, weak head normalization and careful handling of variable binding and scope. Language workbenches provide abstractions that address these challenges. Core concerns such as name resolution, binding structure, and unification are treated as first-class components, allowing language designers to focus on the semantics of their type systems rather than low-level implementation details.

Spoofax (Wachsmuth, Konat, and Visser 2014), is a language workbench developed at TU Delft. Type checking in Spoofax is done using Statix (Antwerpen et al. 2018), a constraint-based language designed for expressing typing and name resolution rules declaratively. The main contrib ution of Statix is name resolution through scope graphs, which are graph-based representation of scopes and the relations between them. Scope graphs are particularily well suited for imports, mutual refferences and other types of non-lexical scoping. Since scope graphs are used to model scoping, it is not possible to create function terms in Statix. Likewise, the semnatics of substitution must be explicitely encoded by the programmer.

On the other hand, other approaches to name binding have been studied. Name binding can be represented more closely to lambda calculus via higher order abstract syntax (HOAS) (Pfenning and Elliott n.d.). The core idea of HOAS is that functions (lambda abstractions) are first class terms. They can be created by the user and they can also be deconstructed. A system implementing HOAS will give the user the ability to go under binders, by generating fresh placeholders for the bound name. Besides generating fresh names, a HOAS system will provide the implication operator that acts as an implicit context. An expression of the form $P \Rightarrow F$ can be viewed as "If predicate P holds, then prove F". This paradigm can be used for typechecking, to emulate typing contexts very closesly to their formal definition: "Assuming that variable X has type A, prove the body has type B".

HOAS and Scope Graphs differ fundamentally in how they treat binding and unification. Since HOAS follows the syntax of simply typed lambda calculus, names are bound only by constructing a lambda abstraction while properties are asserted via implication. In return, users get capture-avoiding substitution, lexical scoping and higher order unification for free. While scope graphs provide strong support for modular and non-lexical scoping, they do not natively provide higher-order pattern unification, which is central to dependently typed systems.

This friction motivates the central question of this thesis: can higher-order pattern unification be integrated with a scope-graph-based constraint system in a principled and practical way (RQ1)? Furthermore, how does such an approach compare to higher-order logic programming systems when implementing a dependently typed language (RQ2)?

To address these questions, we design and evaluate a higher-order pattern unification algorithm tailored to a scope-graph-based setting, and analyze its implications for implementing dependently typed languages in Spoofax.

1.1 Contributions

This thesis addresses two research questions and makes the following contributions.

Higher-order pattern unification in scope-graph systems. We design and formalize an algorithm for higher-order pattern unification in a constraint-based system that uses scope graphs for name binding. To validate the algorithm, we implement a minimal dependently typed core language in Statix, demonstrating that higher-order pattern unification can be integrated with scope-graph-based name resolution and supports core dependent type features.

Comparison of implementation paradigms for dependent typing. We provide a structured comparison between higher-order logic programming systems such as λ-Prolog and scope-graph-based constraint systems. The analysis examines differences in expressivity and computability, including the treatment of binding, fresh name generation, modularity, and non-lexical scoping. This clarifies the trade-offs between meta-level binding mechanisms and explicit scope representations when implementing practical dependently typed languages.

Chapter 2

Background

In this chapter we will argue about the importance of rigouros type systems, such as dependent type theory. We will also explain the unique binding requirements of this system and how it can be used to provide better editor services. Furthermore, we explain the theory behind two approaches to name binding: scope graphs and higher order abstract syntax.

A lot of effort has been made towards improving type safety of new languages. The benefits of strong type systems are easy to see: self-documenting code, more bugs prevented at compile time and stronger invariants. The latter is becoming increasingly important. If an identifier has a type, the programmer can make certain assumptions about its value. For example, in Rust, an invariant of reference values is that they can never be dangling. This property is guaranteed at compile time by the borrow checker. Furthermore, lifetime annotations in struct definitions such as:

```
struct A<'a> {
  pub s: string<'a>
}
```

guarantee that the heap-allocated string values lives as long as the struct it belongs to, making it safe to acces. Another invariant programmers might be familiar with are (non)-nullable values. Languages like TypeScript and Kotlin can perform type narrowing after the value is asserted as null or non-null. The type of a nullable value `x: int?`, can be narrowed as so:

```
if x != null {
  // x guaranteed to be int and not null here
}
```

The compiler's ability to check invariants strongly correlates with the safety of the code. However, in most conventional type systems, the set of invariants that can be checked is fixed by the compiler's implementation and users have no way to extend it.

2.1 Dependent Types

Dependent type systems were designed to address this challenge. In a dependently typed language, types may depend on values, enabling programmers to express and verify arbitrary invariants within the type system itself.

2.1.1 Function types and Pi types.

In the simply typed calculus, a function type $A \to B$ describes functions that take an argument of type A and return a result of type B. The return type B is fixed and cannot refer to the value of the argument. A *dependent function type* (or *Pi type*), written $\Pi\,(x : A).\, B(x)$

generalises this: the return type $B(x)$ may mention the argument x. When the return type does not depend on the argument, the Pi type reduces to an ordinary function type $A \to B$.

As a first example, consider a function `default` : $\Pi\,(A : \text{Type}).\, A$ that returns a default value for a given type. The return type A refers to the value of the first argument. This cannot be expressed with an ordinary function type.

2.1.2 Implicit arguments.

In practice, many type arguments can be inferred from context. Most dependently typed languages allow such arguments to be marked as *implicit*, meaning they are automatically filled in by the type checker during unification. For example, the identity function can be given the type id : $\{A : \text{Type}\} \to A \to A$, where the braces indicate that A is implicit. The caller writes id 42 rather than id $\mathbb{N}$ 42, and the type checker infers that $A = \mathbb{N}$.

2.1.3 Data types and constructors.

Dependently typed languages support *indexed* data types, whose type signature carries information about the values it classifies. A data type may have *parameters*, which are fixed across all constructors, and *indices*, which may vary between constructors. This distinction is important: indices allow data types to be indexed by values, which is the key mechanism for encoding invariants in the type system.

It is often easier to think of types as propositions that say something about a class of values. If we can provide a value for a type, the type checker will assert the invariant described in the type. As an example, we can design an invariant which guarantees that a value is non-null. We will use the Option type to represent *null* as it is the idiomatic way to represent nullable values in pure languages.

Listing 2.1: Non-null invariant written in Agda

```
data Option (A : Set) : Set where
  null : Option A
  some : A → Option A

-- Non-null invariant expressed as a type over Option values
-- Only values constructed with some will make this proof true
data IsNonNull {A : Set} : Option A → Set where
  is-non-null : ∀ {x} → IsNonNull (some x)

-- the caller must provide a proof of non-nullity
unwrap : {A : Set} → (m : Option A) → IsNonNull m → A
unwrap (some x) is-non-null = x

is-non-null :: IsNonNull (some 42) -- will compile
is-non-null :: IsNonNull null -- won't compile because the type has 0 inhabitants
```

In this example, the non null invariant is required by the `unwrap` function. The type checker will accept the expression `unwrap x proof` in two cases: a) `x` is constructed with `some` at the call site or b) we receive the proof that `x` is not null from another function.

The most important feature of dependent types, that allows us to use values (or any terms for that matter) at compile time is the particular name binding structure. In the previouse example, `unwrap` can use the **value** of its first argument in the type signature, by binding it to the name m.

2.2 Name binding

This section will give an overview of name bindings in the context of static analysis. We will also discuss the relation between scoping and name binding.

In the literature, there are many types of name binding, but this work will focus on static name binding (binding that happens at compile time). In static analysis of programs it is common to infer information about a name when it is first introduced (*declared*), while later *refferences* of the identifier can re-use the collected information or refine it (van Antwerpen 2024). In a well formed program all refferences must be *bound* to a single declaration. However, in many programming languages used in practice, it is not always obvious how to map a refference to its declaration. This process is known as *name resolution* (van Antwerpen 2024).

A common way to represent the result of name resolution is through *def-use chains*, which link each reference directly to its corresponding declaration (Aho et al. 2006). Once established, def-use chains form the backbone of many compiler analyses and IDE services such as go-to-definition, find references, and rename refactoring.

The rules that govern name resolution are determined by the scoping discipline of the language. The most common discipline is *lexical scoping* (also known as *static scoping*), where the scope of a name is determined by its position in the AST. In a lexically scoped language, an inner scope may *shadow* a declaration from an enclosing scope, and resolution proceeds outward from the reference site until a matching declaration is found. Lexical scoping was popularised by Algol 60 and is used by the majority of modern programming languages (Scott 2015). However, many practical languages also feature *non-lexical* scoping mechanisms that cannot be explained solely by the AST hierarchy. Examples include module imports, class inheritance, qualified names, and forward-references. Non-lexical scoping increases the complexity of typechecking. Since the scoping rules don't follow the hierarchy of the AST neither can the type checker. As an example, to typecheck forward references, the type checker cannot simply follow the AST, as it will encounter references of names that have not yet been declared. To account for this it should collect and process all available declarations within a scope before proceeding. It follows that scoping and name resolution are tightly coupled with the operational semantics of a type checker. In most languages, non-lexical scoping is handled in an ad-hoc manner embedded in the type checker and other static analyses (Neron et al. 2015).

2.3 Name binding in dependent types

A dependently typed language adds subtle complexities to the name binding and resolution process. One radical difference to a simply typed language is the ability to bind values in types. When performing an application $f\ x$, the value of x might be bound by f's binder and by f's type.

Another challenge of name binding comes with inference. In dependently typed languages inference is complex and requires higher-order unification. Most implementations will create ”holes” (metavariables) where a term must be inferred. Here, term refers to either a value or a type, since in this type system we can infer both. Readers might be familiar with Hindley-Milner type systems, where all types are inferrable. Such languages will insert a metavariable in place of the type of a definition/variable. When performing static analysis we are able to either infer or check a term, but metavariables have a different lifecycle. When a metavariable is defined, there is no information associated with them (perhaps only their type). Later use of terms with metavariables will generate unification constraints, which in turn allow the typechecker to set a **value** for the ”hole”. Language implementations view type checking and constraint solving as separate systems. The interface between the two components can look as follows:

1. The solver can create new metavariables (by itself or when requested)
2. Typechecker can publish a constraint for unifying two terms
3. Typechecker can give a term with "holes" and receive back a term with metavariables substituted for their current solutions (if they exist) from the solver

Point 2. is perhaps vague. Does the type checker expect a response from the solver or does it just move on? This question motivates the existence of another approach for unification called *dynamic pattern unification*. The dynamic approach distinguishes between 3 states for constraints: solvable, impossible to solve or delayed. The latter suggests that constraints that come later might help us solve constraints we accumulated earlier. Furthermore, unification becomes decoupled from the AST and requires a form of non-lexical scoping. We need to capture the required context (bindings, telescopes) for a constraint and store it.

As highlighted by point 3, the type checker needs to know information about a metavariable. We make a distinction between the scope in which a metavariable is created and the scope in which it is solved. If we strictly follow lexical scoping, the scope in which we query might not be a descendant of the scope in which we instantiate the metavariable. Therefore, metavariables and their solutions need to live in a global, mutable scope.

2.4 Scope Graphs

Modern programming languages combine multiple scoping mechanisms, including lexical scoping induced by syntactic structure and non-lexical scoping arising from constructs such as imports, inheritance, or other visibility relations. Traditional formulations of name resolution, for instance based on hierarchical symbol tables, are well-suited for purely lexical scoping but become increasingly ad hoc when extended to accommodate such non-lexical mechanisms. Scope graphs provide a uniform representation in which both forms of scoping are modeled within a single formalism, by representing scopes as nodes in a graph and scoping relations as labeled edges.

Formally, a scope graph is a directed labeled graph $G = (S, E, D)$, where S is a set of scopes, D is a set of declarations, and $E \subseteq S \times L \times S$ is a set of labeled edges between scopes, for some set of labels L. Each declaration $d \in D$ is associated with a scope in S and introduces a name. References to names occur in scopes and must be resolved to declarations. Lexical scoping is typically represented by edges corresponding to syntactic nesting, whereas non-lexical scoping mechanisms, such as imports or inheritance, are represented in the same graph as additional labeled edges (Antwerpen et al. 2018). This representation abstracts from the concrete language constructs and instead captures name binding in terms of reachability in the graph.

Name resolution is defined as a path search problem over the scope graph. Given a reference to a name x in a scope $s \in S$, resolution amounts to finding a declaration d of x in some scope s' such that there exists a path from s to s' that satisfies a given resolution policy. Such a policy constrains the admissible sequences of edge labels and typically induces a preference ordering over paths in order to model shadowing. In particular, declarations that are reachable via preferred paths (for example, shorter paths or paths following specific edge labels) take precedence over others, ensuring that more local declarations shadow those that are more distant in the graph.

In *Statix*, scope graphs are integrated into a constraint-based framework in which both the construction of the graph and the resolution of names are expressed declaratively. Rather than constructing the scope graph procedurally, one specifies constraints that introduce scopes, declarations, and edges, as well as constraints that require certain references to resolve to declarations. Concretely, scope construction is expressed using constraints that generate fresh

scopes, introduce declarations of names in scopes, and relate scopes via labeled edges. Name resolution is expressed by constraints of the form $\texttt{resolve}(s, x) = d$, requiring that the reference to x in scope s resolves to declaration d according to the resolution policy induced by the scope graph.

An advantage of Statix is that scope graph constraints interact with other semantic constraints, such as equality or typing constraints. For example, resolving a reference yields a declaration whose associated semantic information, such as a type, may be subject to further constraints. As a result, name resolution is not treated as a separate phase but is instead integrated into a unified constraint system.

Constraint solving proceeds by incrementally refining a partial scope graph and a set of constraints. Resolution constraints are solved only when the relevant portion of the scope graph is sufficiently closed to guarantee a stable result (i.e. Statix knows we won't extend that scope in the future). Otherwise, they are suspended. As additional scopes, edges, and declarations are introduced, suspended resolution constraints may become solvable. Therefore, name resolution is specified as a global consistency condition rather than an eager operation. Moreover, this semantics allow a typechecker to deal with non-lexical scoping such as forward refferences. We can issue a constraint that a reference should be resolvable before we create a declaration for that name. The constraint should be delayed until we encounter the declaration.

2.4.1 Statix

Statix is a constraint-based language used in the Spoofax language workbench for specifying type systems and name resolution declaratively. It builds on scope graphs to provide a unified framework in which both binding structure and typing constraints are expressed as logical constraints.

In Statix, programs are analyzed by generating and solving constraints over terms and scope graphs. Terms represent syntactic entities of the object language, while constraints capture relationships between them, such as equality, typing requirements, or name resolution conditions. Constraint solving proceeds incrementally, refining both the term structure and the scope graph until a consistent solution is found.

The principal design choice in Statix is that terms are first-order: they are constructed from user-defined constructors and variables, but do not include higher-order constructs such as lambda abstractions. As a result, binding structure is not represented within terms themselves, but externally via scope graphs. Likewise, unification in Statix is restricted to first-order terms.

This separation enables flexible modeling of complex scoping mechanisms, including non-lexical scoping.

2.4.2 Another way to model name binding: Higher-Order Abstract Syntax

An alternative approach to modeling name binding is given by Higher-Order Abstract Syntax (HOAS), in which binding constructs of an object language are represented using the binding mechanisms of a meta-language (host language). Instead of representing scopes, declarations, and references explicitly, HOAS encodes binding structure directly using functions of the meta-language.

The key idea is to represent object-language binders as meta-level functions. A bound variable is represented by a meta-level variable introduced by a function argument, and occurrences of that variable are represented by the same meta-level variable. As a result, substitution and α-equivalence are handled implicitly by the meta-language, without requiring explicit definitions of these operations.

For example, a λ-abstraction can be represented as a meta-level function:

```
lam (x \ body)
```

Here, x is a meta-level variable representing the bound variable, and $body$ is a term in which x may occur. Application is represented as:

```
app (lam (x \ body)) arg
```

HOAS can be embedded in a variety of meta-languages, such as Agda or Coq, where binders are represented using higher-order functions. However, these systems rely on an intensional (or weak) function space, in which functions are treated as opaque values that cannot be inspected or decomposed by pattern matching. As a result, while HOAS representations can be constructed, it is difficult to analyze or invert them, since the structure of a function cannot be recovered by pattern matching. For this purpose, logic programming languages are easier to use with HOAS. Implementations such as ELPI (Dunchev et al. 2015) allow users to inspect functions by going under binders. The semantics of a logic programming language are slightly different. Programs are expressed as relations rather than functions, and evaluation proceeds by attempting to prove goals via unification and backtracking. ELPI uses higher-order pattern unification on meta-level representations of object-language terms, including those containing binders. Unification is restricted to the higher-order pattern fragment, where free variables may only be applied to distinct bound variables, to preserve decidability.

A typical use case in ELPI is the representation of typing rules. For example, the typing rule for lambda abstraction can be encoded as:

```
check (lam Body) (arrow A B) :-
  (pi x \ type_of x A => check (Body x) B).
```

In this encoding, x is introduced as a fresh meta-level variable. The implication operator `=>` extends the context with the assumption that x has type A when checking the body. The meta-language ensures that x is correctly scoped, and that occurrences of x in $Body$ refer to the same entity.

Similarly, application can be expressed as:

```
infer (app F X) B :-
  infer F (arrow A B),
  check X A.
```

This representation contrasts with first-order encodings, where explicit environments or name management must be introduced. In HOAS, these aspects are delegated to the meta-language. The difference in how binding structures can be inspected and manipulated influences the design of type checkers and unification algorithms.

Chapter 3

Results

To demonstrate the combination of scope graph name resolution with higher order unification, we will describe how to typecheck a dependent type language. The main feature of this language is the ability to infer implicit arguments.

$$
\begin{array}{lll}
AB & ::= Set \mid A \equiv B \mid A \to B \mid (x : A) \to B \mid \{x : A\} \to B & types \\
MN & ::= x \mid ?m \mid M :: A \mid \lambda x.\ M \mid M\ N \mid refl \mid let\ x = M;\ N \mid postulate\ x\colon A;\ M & terms \\
\Gamma & ::= \Gamma; x\colon A & typing\ context \\
\Delta & ::= \Delta; ?m\colon A & meta - context
\end{array}
$$

Typechecking is done with bidirectional rules with $\Rightarrow$ for inferring Figure 3.1 and $\Leftarrow$ for checking Figure 3.2. For some rules the relation $A = B$ appears, which suggests that A and B should be unified. The meta context Δ should have mutable semantics, meaning that solving or creating a metavariable in one branch of the inference tree should mutate the context for all the other branches. This is opposed to the typing context, which is responsible for storing the identifiers that are in scope, associated with their types.

$$
\textsc{Meta}\ \frac{?a : A \in \Delta}{\Gamma, \Delta \vdash ?a \Rightarrow A} \qquad \textsc{Var}\ \frac{x : A \in \Gamma}{\Gamma, \Delta \vdash x \Rightarrow A} \qquad \textsc{Set}\ \frac{}{\Gamma, \Delta \vdash Set \Rightarrow Set}
$$

$$
\textsc{Pi}\ \frac{\Gamma, \Delta \vdash t \Rightarrow (x : A) \to B \qquad \Gamma, \Delta \vdash u : A}{\Gamma, \Delta \vdash t\ u \Rightarrow B[x/u]}
$$

$$
\textsc{ImpPi}\ \frac{\Gamma, \Delta \vdash t \Rightarrow \{x : A\} \to B \qquad \Delta; ?a \vdash (t\ ?a)\ u \Rightarrow C}{\Gamma, \Delta \vdash t\ u \Rightarrow C} \qquad \textsc{Let}\ \frac{\Gamma, \Delta \vdash t \Rightarrow A \qquad \Gamma; x\colon A, \Delta \vdash u : B}{\Gamma, \Delta \vdash let\ x = t;\ u \Rightarrow B}
$$

$$
\textsc{Postulate}\ \frac{\Gamma, \Delta \vdash A \Rightarrow Set \qquad \Gamma; d\colon A, \Delta \vdash u : B}{\Gamma, \Delta \vdash postulate\ d\colon A;\ u \Rightarrow B}
$$

Figure 3.1: Inference rules

$$\textsc{Lam}\ \frac{\Gamma; x\colon A, \Delta \vdash t \Leftarrow B}{\Gamma, \Delta \vdash \lambda x.\ t \Leftarrow (x : A) \to B} \qquad \textsc{Implicit}\ \frac{\Gamma, \Delta; ?m : A \vdash t \Leftarrow B[x/?m]}{\Gamma, \Delta \vdash t \Leftarrow \{x : A\} \to B} \qquad \textsc{Refl}\ \frac{\Gamma, \Delta \vdash X = Y}{\Gamma, \Delta \vdash refl \Leftarrow X \equiv Y}$$

$$\textsc{Set}\ \frac{}{\Gamma, \Delta \vdash Set \Leftarrow Set} \qquad \textsc{Pi-Wellformed}\ \frac{\Gamma, \Delta \vdash A \Leftarrow Set \qquad \Gamma; x : A, \Delta \vdash B \Leftarrow Set}{\Gamma, \Delta \vdash (x : A) \to B \Leftarrow Set}$$

$$\textsc{ImpPi-Wellformed}\ \frac{\Gamma, \Delta \vdash A \Leftarrow Set \qquad \Gamma; x : A, \Delta \vdash B \Leftarrow Set}{\Gamma, \Delta \vdash \{x : A\} \to B \Leftarrow Set} \qquad \textsc{Eq-Wellformed}\ \frac{\Gamma, \Delta \vdash t \Rightarrow A \qquad \Gamma, \Delta \vdash u \Rightarrow B \qquad A = B}{\Gamma, \Delta \vdash t \equiv u \Leftarrow Set}$$

$$\textsc{infer}\ \frac{\Gamma, \Delta \vdash t \Rightarrow A' \qquad A = A'}{\Gamma, \Delta \vdash t \Leftarrow A}$$

Figure 3.2: Checking rules

3.0.1 Meta-context in scope graph

To model the meta-context in the scope graph framework we instantiate a toplevel meta-scope called `metaS`. Each rule should be given a reference to this exact scope, such that it has permission to extend it. To model the creation of new unsolved metavariable we create a new scope s' with an M labeled edge to `metaS`. The reference to the scope s' becomes the unique identifier of the metavariable. When we want to instantiate a metavariable, we will create the `inst[id, V]` relation in metaS, where `id` is the metavariable's identifier and V represents its value. Such a relation can be easily queried for in any branch of our inference. Since we always have a reference to `metaS`, we want to search for `inst` relations with a given id, without leaving `metaS`.

Chapter 4

Related work

Related work here.

Chapter 5

Conclusion

Conclusion here.

Bibliography

Aho, Alfred V. et al. (2006). *Compilers: Principles, Techniques, and Tools*. 2nd ed. Addison-Wesley. ISBN: 978-0-321-48681-3.

Antwerpen, Hendrik van et al. (Oct. 24, 2018). "Scopes as types". In: *Case Studies for Article: Scopes as Types* 2 (OOPSLA), 114:1–114:30. DOI: 10.1145/3276484. URL: https://dl.acm.org/doi/10.1145/3276484 (visited on 01/09/2026).

Dunchev, Cvetan et al. (Nov. 2015). "ELPI: fast, Embeddable, λProlog Interpreter". In: *LNCS*. Suva, Fiji. URL: https://inria.hal.science/hal-01176856 (visited on 02/09/2026).

Erdweg, Sebastian et al. (Dec. 1, 2015). "Evaluating and comparing language workbenches: Existing results and benchmarks for the future". In: *Computer Languages, Systems & Structures*. Special issue on the 6th and 7th International Conference on Software Language Engineering (SLE 2013 and SLE 2014) 44, pp. 24–47. ISSN: 1477-8424. DOI: 10.1016/j.cl.2015.08.007. URL: https://www.sciencedirect.com/science/article/pii/S1477842415000573 (visited on 03/23/2026).

Johansson, Marcus and Jesper Lloyd (n.d.). "Eliminating the problems of hidden-lambda insertion". In: ().

Neron, Pierre et al. (2015). "A Theory of Name Resolution". In: *Programming Languages and Systems (ESOP 2015)*. Vol. 9032. Lecture Notes in Computer Science. Springer, pp. 205–231. DOI: 10.1007/978-3-662-46669-8_9.

Pfenning, Frank and Conal Elliott (n.d.). "Higher-Order Abstract Syntax". In: ().

Reis, Giselle (n.d.). "Curry-Howard Correspondence". In: ().

Scott, Michael L. (2015). *Programming Language Pragmatics*. 4th ed. Morgan Kaufmann. ISBN: 978-0124104099.

van Antwerpen, H. (2024). "Declarative Name Binding for Type System Specifications". English. Dissertation (TU Delft). Delft University of Technology. ISBN: 978-94-6384-704-9. DOI: 10.4233/uuid:4bf44aa1-779c-4a96-8c55-5e1b54e16119.

Wachsmuth, Guido H., Gabriël D.P. Konat, and Eelco Visser (2014). "Language Design with the Spoofax Language Workbench". In: *IEEE Software* 31.5, pp. 35–43. DOI: 10.1109/MS.2014.100.

Acronyms

AST abstract syntax tree

DSL domain-specific language

Appendix A

A

Appendix here.